

SHF : Funnel Shift

Format

```
SPA 5.0:
    {@{!}Pg}    SHF.dir{.mode}{.maxshift}{.xmode}  Rd{.CC}, Ra, Sb, Rc  {&req_6}  {?sched}  ;

.dir:          {.R, .L}
               .R (Right): Shift [Rc, Ra] right Sb bits
               .L (Left) : Shift [Rc, Ra] left  Sb bits.

.mode:         {.W, .C*}
               .C (Clamp ): Sb is treated as an unsigned integer and is clamped (0-.maxshift).
               .W (Wrap  ): Sb is treated as an unsigned integer and is masked by (.maxshift -1).

.maxshift:     {.32*, .U64, .S64 }
               .32 : max shift is 32 bits.
               .S64: max shift is 64 bits. Also indicates that signed (arithmetic) shift should be used
               .U64: max shift is 64 bits. Also indicates that unsigned (logical) shift should be used
               .S64 can only be specified with .dir = .R (arithmetic right shift)

.xmode:        { < NULL >*, .X, .HI, .XHI }
               .X (must be used with .CC) accumulates zero flag, preserves other flags (CF/OF/SF)
               .XHI (must be used with .CC) accumulates zero flag, preserves CF, sets OF and SF.
               It also has an effect of shifting by 32 more bits than derived from Sb/.mode.
               .HI has the effect of shifting by 32 more bits than derived from Sb/.mode.
               If used with .CC, instruction sets all the flags.
               If just .CC is used without any .X/.XHI, instruction sets all the flags
               Use .X and .XHI to update the condition code flags for extended precision multi-word shifts.
               Use .HI/.XHI is allowed with only .dir=.R only.

.CC:           Write condition codes based on result Rd

SHF allows the following for source Sb:
    Sb(register)
    Sb(unsigned 6-bit integer immediate IMMU6)
```

Description

SHF funnel shifts a 64-bit concatenation of two registers yielding a 32-bit result, to implement extended precision multi-word shift and rotate operations.

SHF.R shifts the 64-bit concatenation of [Rc, Ra] right by shift amount derived from Sb, extracts the 32 **least-significant** bits, and writes the result to Rd. Rc holds bits 63:32 and Ra holds bits 31:0 of the 64-bit source. Shift amount is specified in Sb and depending upon the clamping/wrapping specified by .mode, varies between 0..maxshift. The equivalent C expression is:

```
U32 shift = (.mode == .W) ? ( Sb & (maxshift -1)) : min ((unsigned) Sb, maxshift);
U64 val   = (Rc << 32 | Ra);
U32 Rd    = (.maxshift == .S64) ? ( ((S64)val >> shift) & 0xffff_ffff) :
           ( ((U64)val >> shift) & 0xffff_ffff );
```

SHF.L shifts the 64-bit concatenation of [Rc, Ra] left by shift amount derived from Sb, extracts the 32 **most-significant** bits, and writes the result to Rd. Rc holds bits 63:32 and Ra holds bits 31:0 of the 64-bit source. Shift amount is specified in Sb and depending upon the clamping/wrapping specified by .mode, varies between 0..maxshift. The equivalent C expression is:

```
U32 shift = (.mode == .W) ? ( Sb & (maxshift -1)) : min ((unsigned) Sb, maxshift);
U64 val   = (Rc << 32 | Ra);
val       = (val << shift) ;
U32 Rd    = (val >> 32) ;
```

The default mode .C clamps the Sb value to 0 - maxshift, while mode .W masks the Sb value with (maxshift -1).

Adding the .CC suffix to destination Rd writes the condition code CC register. The .X suffix with .CC writes CC with an extended precision multi-word update rather than writing CC based only on the result value.

Use funnel shift for 64-bit and larger multi-word shift operations and for rotate operations. A left funnel shift by Sb is equivalent to a right funnel shift by (32 - Sb), and conversely. To shift 64 bits or more to the right, use SHF.R to extract the 32 lsbs of 64-bits shifted right. Use the SHR instruction to generate the most-significant result word with zero or sign fill. To shift 64 bits or more to the left, use SHL for the least significant word and use SHF.L for the more significant words. For 32-bit rotate operations, place the same value in Ra and Rc.

When using the .CC and .X modes, an extended-precision right shift is performed by processing words in order from least-significant to most-significant. At the end of the extended-precision shift sequence, the condition code flags reflect the overall multi-word result. The carry flag is set by the initial SHF.R.X Rd.CC instruction, and sign and overflow flags are set by the final SHR.XHI Rd.CC instruction. The zero flag value is accumulated by shift instructions in .X and .XHI mode. See example at bottom.

Examples:

```

# bitextract 32 bits from a 64 bit data
SHF.R      R0, R0, R8,  R1;      # extract 32 lsbs of [R1,R0] shifted right by R8
# rotate right
SHF.R      R9, R9, 13,  R9;      # rotate R9 right 13 bits
# rotate left
SHF.L      R9, R9, R8,  R9;      # rotate R9 left by shift amount in R8

# emulate SHL(.C,.W)
SHF.L.(C/W).32      R10, RZ, R8, R4;      # dest(R10) = [R4, 0] << (shift=R8)
# emulate SHR.S32
SHF.R.(C/W).(S64/32).HI R10, RZ, R8, R4;      # dest(R10) = [R4, 0] >> (shift=R8)
# emulate SHR.U32
SHF.R.(C/W).32      R10, R4, R8, RZ;      # dest(R10) = [0 ,R4] >> (shift=R8)

# 2 instruction emulation of a unsigned (logical) shift right of 64 bit data [R1,R0] >> R6
# set proper CC flags, R6 has shift amount.
SHF.R.(C/W).(U64/32) R0.CC, R0, R6, R1;      # funnel shift [R1,R0] right by R6, get 32 lsbs. Set all flags. Zero flag used next instru
SHR.U32.C.XHI      R1.CC, R1, R6;      # get 32 msbs of result. chain in zero flag from SHF. set sign/overflow

# sequence for same logical right shift as above, but using only SHF
SHF.R.(C/W).(U64/32) R10,  R4, R8, R5;      # dlo(R10)  = [R5, R4] >> (shift=R8)
SHF.R.(C/W).U64.XHI R11,  RZ, R8, R5;      # dHi(R11)  = [R5,  0] >> (shift=R8)

# 2 instruction emulation of a signed (arithmetic) shift right of 64 bit data [R1,R0] >> R6
# set proper CC flags, R6 has shift amount.
SHF.R.(C/W).S64      R0.CC, R0, R6, R1;      # funnel shift [R1,R0] right by R6, get 32 lsbs. Set all flags. Zero flag used next instru
SHR.S32.C.XHI      R1.CC, R1, R6;      # get 32 msbs of result. chain in zero flag from SHF. set sign/overflow

# sequence for same arithmetic right shift as above, but using only SHF
SHF.R.(C/W).S64      R10,  R4, R8, R5;      # dlo(R10)  = [R5, R4] >> (shift=R8)
SHF.R.(C/W).S64.XHIR11, RZ, R8, R5;      # dHi(R11)  = [R5,  0] >> (shift=R8)

# 2 instruction emulation sequence for shift left of 64 bit data [R1,R0] << R6
# set proper CC flags, R6 has shift amount.
SHF.L.(C/W).(U64/32) R1.CC, R0, R6, R1;      # funnel shift [R1,R0] left by R6, get 32 msbs. Set all flags. Zero flag used next instruc
SHL.C.X      R0.CC, R0, R6;      # get 32 msbs of result. chain in zero flag from SHF. set sign/overflow

# sequence for same SHL.64 as above, but using only SHF
SHF.L.U64      R11,  R4, R8, R5;      # dHi(R11)  = [R5, R4] << (shift=R8)
SHF.L.U64.X R10, RZ, R8, R4;      # dLo(R10)  = [R4,  0] << (shift=R8)

# wide shift right (say 128 bit datatype)      [R3,R2,R1,R0] >> 11
# In general for multi-word right shifts, start construction the result from right-most (least-significant) word of result with SHF.R,
# moving to left, accumulating zero flag via SHF.R.X while keeping other flags intact. Finally use an SHR.XHI to commit the final update
SHF.R.(C/W).(U64/32) R4.CC, R0, 11, R1;      # funnel shift [R1,R0] right by 11, get 32 lsbs. Set all flags. Zero flag used next instru
SHF.R.(C/W).(U64/32).X R5.CC, R1, 11, R2;      # funnel shift [R1,R0] right by 11, get 32 lsbs. accumulate zero flag
SHF.R.(C/W).(U64/32).X R6.CC, R2, 11, R3;      # funnel shift [R1,R0] right by 11, get 32 lsbs. accumulate zero flag
SHR.{S32/U32}.C.XHI R7.CC, R1, 11;      # get 32 msbs of result. chain in zero flag from SHF. set sign/overflow

# wide shift left (say 128 bit datatype)      [R3,R2,R1,R0] << c[0][0]
# In general for multi-word left shifts, start construction the result from left-most (most-significant) word of result with SHF.L,
# moving to right, accumulating zero flag via SHF.L.X while keeping other flags intact. Finally use an SHL.XHI to commit the final update
SHF.L.(C/W).(U64/32) R7.CC, R2, R10, R3 ;      # funnel shift [R3,R2] left by R10, get 32 msbs. Set all flags. Zero flag used next instru
SHF.L.(C/W).(U64/32).X R6.CC, R1, R10, R2 ;      # funnel shift [R2,R1] left by R10, get 32 msbs. accumulate zero flag
SHF.L.(C/W).(U64/32).X R5.CC, R0, R10, R1 ;      # funnel shift [R1,R0] left by R10, get 32 msbs. accumulate zero flag
SHL.C.X      R4.CC, R0, R10 ;      # get 32 msbs of result. chain in zero flag from SHF. set sign/overflow

```

[back to index of instructions](#)